

TOWARDS FEDERATED LEARNING AT SCALE: SYSTEM DESIGN

In this paper, the authors present a high level system design for federated learning and the potential challenges that come with it. The authors focus on support for synchronous federated learning, while mitigating potential synchronization overhead. The system enables FL through tensor flow on data stored on the device, and the weights are combined in the cloud using federated averaging, constructing a global model which is then pushed back to the devices for inference. The paper addresses several practical issues like device availability, unreliable device connectivity and interrupted execution, lock-step execution across devices with varying availability, and limited device storage and compute resources. These issues are addressed at the communication protocol, device, and server levels.

Devices announce their availability to the server, which then samples and invites a few to work on the FL task by sending instructions to the devices. Each participant runs the FL task based on global state and the local dataset. The server incorporates these updates into the global state and the process repeats. To dictate the rate of task completion, rate steering is introduced, where the server suggests the optimum time window to reconnect to the device. The applications maintain an example store which they use to make their data available to the server. Apart from training tasks, FL plans can also include evaluation tasks. The implementation provides a multi-tenancy architecture supporting training of multiple FL populations of the same task. To protect against the attacks while preserving user anonymity, the model incorporates Android's remote attestation mechanism. The FL server follows the Actor programming model, and handles messages sequentially. Multiple instances of the actors of the same type allows scaling.

Coordinators are top level actors which enable global coordination. Selectors are responsible for accepting and forwarding device connections. Master Aggregators manage the rounds of each FL task by spawning one or more aggregators, to which the work is delegated. No information for a round is written to persistent storage until it is fully aggregated by the Master Aggregator. Selection phase for the next round is run in parallel to the config/reporting phases of the previous round. The architecture also utilizes analytics which do not include PII to monitor devices' health statistics. As aggregation costs grow quadratically with the number of users, aggregators are layered, with each layer aggregating the data from the layer below it.

FL tasks are first simulated over data analogous to on device data, and rely on the FL system's versioning, testing, and deployment infrastructure for automated safety checks. As soon as an FL task has been accepted for deployment, devices checking in may be served the appropriate plan. The work has several applications, like on-device item ranking, content suggestions for on-device keyboards, and next word prediction. The system is scalable and is currently handling a cumulative FL population size of approximately 10M daily active devices, spanning several different applications. The authors note that up to 10k devices participate simultaneously. The number of participating devices depends on the local time of day, with greater number of participants being available during the night. For most models receiving updates from a few hundred devices per FL round is sufficient and the average portion of devices that drop out due to computation errors, network failures, or changes in eligibility varies between 6% and 10%. To accommodate for such failures, the server chooses 130% of the target number of devices to participate. The authors note that this parameter can be tuned based on the empirical distribution of device reporting times and the target number of stragglers to ignore.

A critique of the paper is the use of Android's remote attestation mechanism, also known as the Play Integrity API (which replaces the safetynet API). I have rooted every Android device that I have ever owned, and kernel level root methods like Magisk have the ability to bypass the API - at least for device and basic integrity checks. Although unlocking the bootloader guarantees the failure of strong integrity check, the paper fails to mention the type of integrity needed for the FL tasks. Requiring strong integrity reduces the potential number of devices available for FL, as older devices do not support it altogether, while not requiring it opens up a potential attack vector.